Building a Simple Neural Network using PyTorch

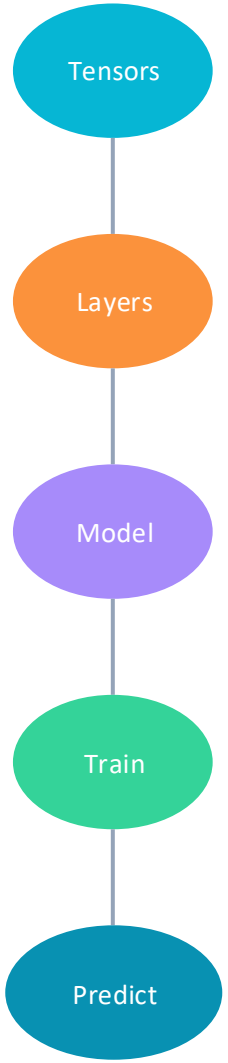
Department of Statistics

Oregon State University

04/29/2026

Today's Roadmap

1. **From Tensors to Neural Network Models**
   How to builds on tensors
2. **Multilayer perceptron**
   Layers, weights, activations
3. **nn.Module**
   PyTorch model class
4. **Building an MLP**
   Multi-layer perceptron from scratch
5. **Training Loop**
   Forward → Loss → Backward → Update
6. **Hands-On Demo**
   Image classification with the MLP



From Tensors to Models

Tensors

Data containers
(like NumPy arrays
with GPU support)

Autograd

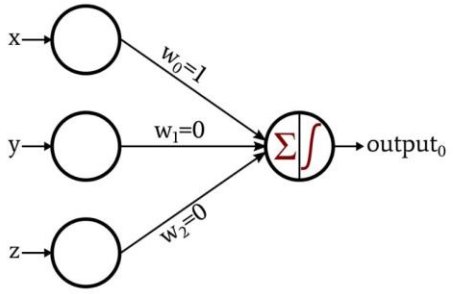
Automatic
differentiation
(computes gradients)

nn.Module

Building blocks
for neural
networks

Multilayer perceptron (MLP)

Perceptron

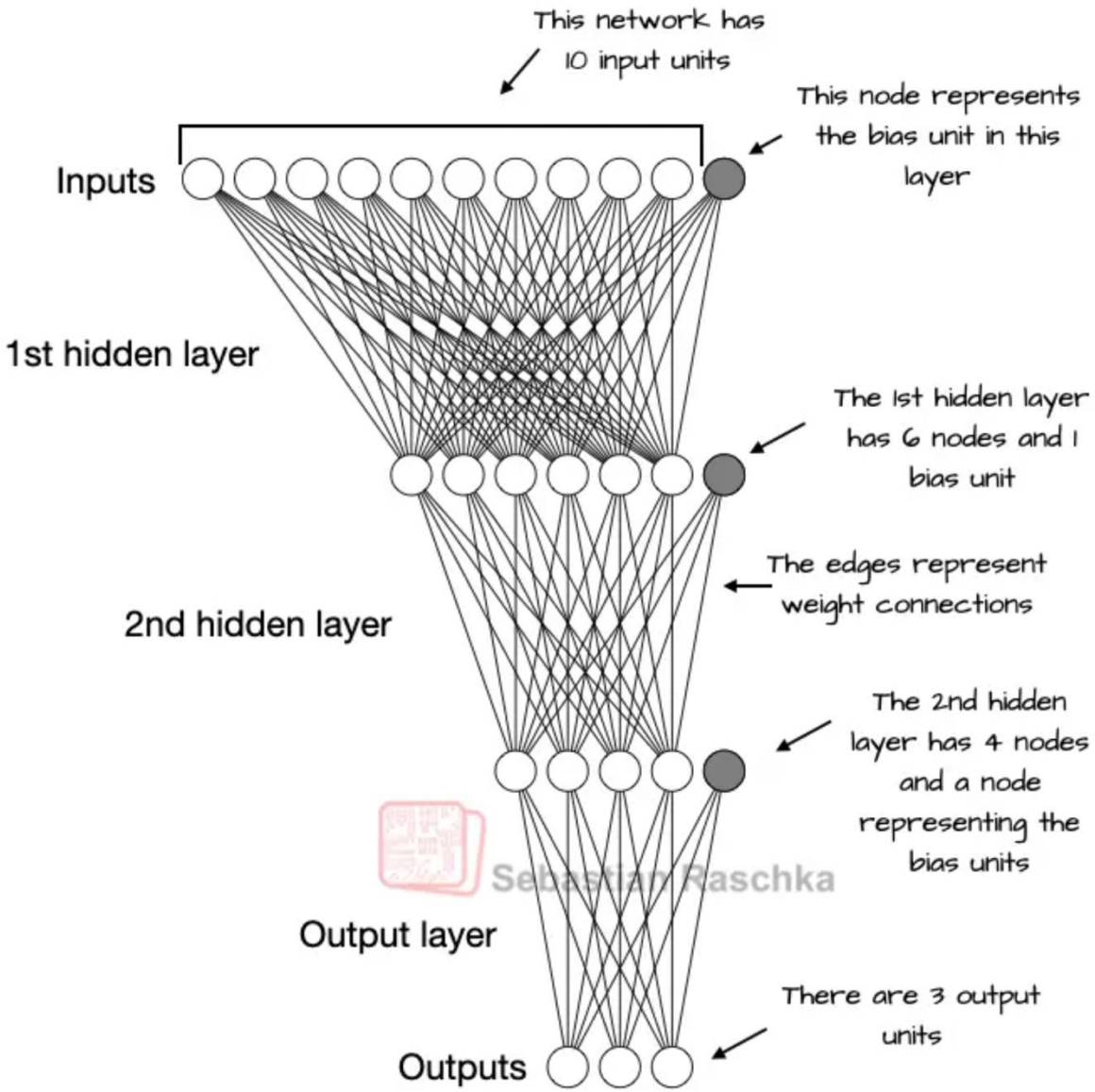


Activation function: sigmoid, tanh, ReLU, ELU etc.

$$y = f(Wx + b)$$

Weights

Bias



Build a neural network model in PyTorch

```
import torch.nn as nn

class MyModel(nn.Module):
    def __init__(self):
        super().__init__()
        # Define layers here

    def forward(self, x):
        # Define computation here
        return x
```

Declare neural network layers

Define forward operations

In PyTorch, **`nn.Module`** is the base class for all neural network models
Define custom class inherited from nn.Module.

Common Building Blocks (layers)

`nn.Linear(in, out)`	Fully connected layer: $y = Wx + b$
`nn.ReLU()`	Activation: $\max(0, x)$
`nn.Sigmoid()`	Activation: $1/(1 + e^{-x})$
`nn.Dropout(p)`	Randomly set some nodes to zero; regularization

Building an MLP from Scratch

```
class SimpleMLP(nn.Module):
    def __init__(self, input_dim, hidden_dim, output_dim):
        super().__init__()
        self.layer1 = nn.Linear(input_dim, hidden_dim)
        self.relu   = nn.ReLU()
        self.layer2 = nn.Linear(hidden_dim, output_dim)

    def forward(self, x):
        x = self.layer1(x)   # Linear transform
        x = self.relu(x)     # Non-linearity
        x = self.layer2(x)   # Output
        return x
```

Model Architecture

Forward pass

Usage

```
model = SimpleMLP(4, 16, 3)
print(model)
output = model(some_tensor)
print(output.shape)
# torch.Size([batch, 3])
```

Training Loop

```
# Setup
model     = SimpleMLP(input_dim=4, hidden_dim=16, output_dim=3)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.01) # Initialize
optimizer


# Training loop
for epoch in range(100):
    # Forward pass
    predictions = model(X_train)
    loss = criterion(predictions, y_train)

    # Backward pass + update
    optimizer.zero_grad()   # Clear old gradients
    loss.backward()         # Compute new gradients
    optimizer.step()        # Update weights

    if epoch % 10 == 0:
        print(f"Epoch {epoch}, Loss: {loss.item():.4f}")
```

- **Loss function:**
  CrossEntropyLoss(): input raw logits
- **Optimizer (lr = learning rate)**
  SGD (Stochastic Gradient Descent)
  Adam (Adaptive Moment Estimation)
  The standard "go-to" optimizer
  AdamW, best for Transformers
- zero_grad() is crucial otherwise gradients would accumulate
- backward() fills .grad for every parameter

Loss Functions & Optimizers

Loss Functions

`nn.MSELoss()` — Regression

Mean Squared Error

`nn.CrossEntropyLoss()` — Multi-class Classification

Input raw logits

`nn.BCELoss()` — Binary classification

Input probabilities

`nn.BCEWithLogitsLoss()` — Binary classification

Input raw logits, more stable, combines a *Sigmoid* layer and the *BCELoss*

Optimizers

`torch.optim.SGD`

Vanilla stochastic gradient descent, can add “momentum” to help escape local minima

`torch.optim.Adam`

Adaptive Moment Estimation , usually the default choice

`torch.optim.AdamW`

Adam + weight decay, better regularization for Transformers

From a logit value, the probability is calculated using the **sigmoid function (i.e. logistic function)**
For multi-classes, the probability is calculated using **softmax** function.